- Design a CNN architecture using frameworks like TensorFlow or PyTorch. For instance:
    - Input Layer: Accepts image data.
    - Convolutional Layers: Extract spatial features using filters.
    - Pooling Layers: Reduce spatial dimensions and computation.
    - Fully Connected Layers: Map extracted features to output categories.
- Use transfer learning with pre-trained models like ResNet, VGG, or EfficientNet for faster training and improved performance.

4. Model Training:
    - Define a loss function, such as categorical crossentropy, and an optimizer like Adam or SGD.
    - Train the model on the training set while validating on the validation set to prevent overfitting.
    - Use callbacks like early stopping and learning rate scheduling to optimize the training process.

5. Performance Evaluation:
    - Evaluate the model using metrics like accuracy, precision, recall, and F1-score on the test set.
    - Generate a confusion matrix to analyze classification errors.
    - Perform error analysis to identify and address misclassifications.

6. Deployment:
    - Deploy the model using frameworks like Flask or FastAPI to create a REST API.